

第六章 STM32L152 中断控制器

6.1 NVIC中断控制器

6.2 NVIC控制器库函数

6.3 EXTI外部中断控制器

6.4 EXTI外部中断控制器库函数

6.5 中断使用示例



6.1 NVIC中断控制器

- 中断指的是当出现需要时，CPU暂时停止当前程序的执行转而执行处理新情况的程序和执行过程；
- 这种处理新情况的程序或执行过程，称为中断服务程序；
- 中断服务程序的入口地址，称为中断向量；
- 中断向量一般存放在一块区域，专门用于存放中断向量，称为中断向量表；
- 存储中断向量的地址，称为中断向量地址。



6.1 NVIC中断控制器

IRQ 编号	优 先 级	优先级类 型	名称	说明	地址
-	-	-	-	保留	0x00000000
	-3	固定	Reset	复位	0x00000004
-14	-2	固定	NMI	不可屏蔽中断	0x00000008
-13	-1	固定	硬件失效 (HardFault)	所有类型的失效	0x0000000C
-12	0	可设置	存储管理 (MemManage)	存储器管理	0x00000010
-11	1	可设置	总线错误 (BusFault)	预取指令失败	0x00000014
-10	2	可设置	错误应用 (UsageFault)	未定义的指令	0x00000018
-				保留	
-5	3	可设置	SVCall	通过SWI指令的系统 服务调用	0x0000002C
-4	4	可设置	调 试 监 控 (DebugMonitor)	调试监控器	0x00000030
				保留	0x00000034
-2	5	可设置	PendSV	可挂起的系统服务	0x00000038
-1	6	可设置	SysTick	系统嘀嗒定时器	0x0000003C
0	7	可设置	WWDG	窗口定时器中断	0x00000040
1	8	可设置	PVD	电压检测中断	0x00000044
2	9	可设置	TAMPER	入侵检测中断	0x00000048
3	10	可设置	RTC	全局定时时钟中断	0x0000004C
4	11	可设置	FLASH	内存全局中断	0x00000050
5	12	可设置	RCC	复位和时钟控制中断	0x00000054
6	13	可设置	EXT0	EXT0线中断	0x00000058
7	14	可设置	EXT1	EXT1线中断	0x0000005C
8	15	可设置	EXT2	EXT2线中断	0x00000060
9	16	可设置	EXT3	EXT3线中断	0x00000064
10	17	可设置	EXT4	EXT4线中断	0x00000068
...	...	...	...	...	...

Exception number	IRQ number	Offset	Vector
16+n	n	0x0040+4n	IRQn
.	.	.	.
.	.	.	.
.	.	.	.
18	2	0x004C	IRQ2
17	1	0x0048	IRQ1
16	0	0x0044	IRQ0
15	-1	0x0040	Systick
14	-2	0x003C	PendSV
13		0x0038	Reserved
12			Reserved for Debug
11	-5	0x002C	SVCall
10			Reserved
9			Reserved
8			Reserved
7			Reserved
6	-10	0x0018	Usage fault
5	-11	0x0014	Bus fault
4	-12	0x0010	Memory management fault
3	-13	0x000C	Hard fault
2	-14	0x0008	NMI
1		0x0004	Reset
		0x0000	Initial SP value

6.1 NVIC中断控制器

- **嵌套向量中断控制器 (Nested Vector Interrupt Controller, NVIC)**
 - 负责中断管理控制，提供可屏蔽的、可嵌套的、具有动态优先级的中断管理。
 - 嵌套向量中断控制器的主要特征如下：
 - 支持最多240种中断，256种中断优先级；
 - 支持中断优先级的动态调整，使用抢占优先级和从优先级管理中断；
 - 支持一个不可屏蔽中断，支持低功耗中断唤醒；
 - 外部IO中断管脚可配置；

6.1 NVIC中断控制器

- **抢占优先级 (pre-emption priority) 和响应优先级 (subpriority) 。**
 - 抢占优先级：高抢占优先级的中断会打断当前用户程序或当前正在执行的低抢占优先级的中断服务程序，即中断嵌套；
 - 从优先级指的是在抢占优先级相同的情况下，高从优先级的中断优先被响应。
 - 在抢占优先级相同的情况下，如果有低从优先级的中断服务程序正在执行，则高从优先级的中断不能打断，即不能嵌套中断。
 - 先如果抢占式优先和从优先级都相等，则根据他们在中断表中的排位顺序决定哪一个先处理。

6.1 NVIC中断控制器

- **抢占优先级（pre-emption priority）和从优先级（subpriority）。**
 - Cortex M3处理器中定义了8个位来设置中断源的优先级，STM32L152实际只使用了其中4个位；
 - 分别有0~4共5组中断优先级方式。
 - 对于分组3的方式，抢占优先级占据3位，则一共有0~7共8个抢占优先级，而从优先级只占1位，则一共只有0~1共2个从优先级。

分组	抢占优先级位数和取值范围	从优先级位数和取值范围
0	0（无）	4（0~15）
1	1（0~1）	3（0~7）
2	2（0~3）	2（0~3）
3	3（0~7）	1（0~1）
4	4（0~15）	0（无）



6.1 NVIC中断控制器

NVIC寄存器

- ST实现的NVIC的特点：
 - 最多81个外部中断【STM32L152RE 57个中断，参见.s文件】
 - 16种优先级
 - 支持电平和边沿信号中断
 - 优先级可调整
 - 支持一个NMI中断

6.1 NVIC中断控制器

NVIC寄存器

- 寄存器说明
- NVIC控制器包含在cortex-M3内核中，最主要的有以下7类32位寄存器
 - 中断使能寄存器 ISER
 - 中断清除寄存器ICER
 - 中断挂起寄存器ISPR
 - 中断挂起清除寄存器ICPR
 - 中断激活寄存器IABR
 - 中断优先级寄存器IPR
 - 软件触发中断寄存器STIR



6.2 NVIC控制器库函数

NVIC寄存器定义

- CMSIS中NVIC控制器的数据结构定义在core_cm3.h中

```
typedef struct
{
    __IO uint32_t ISER[8];          /*!< Offset: 0x000 (R/W) Interrupt Set Enable Register */
    uint32_t RESERVED0[24];
    __IO uint32_t ICER[8];          /*!< Offset: 0x080 (R/W) Interrupt Clear Enable Register */
    uint32_t RSERVED1[24];
    __IO uint32_t ISPR[8];          /*!< Offset: 0x100 (R/W) Interrupt Set Pending Register */
    uint32_t RESERVED2[24];
    __IO uint32_t ICPR[8];          /*!< Offset: 0x180 (R/W) Interrupt Clear Pending Register */
    uint32_t RESERVED3[24];
    __IO uint32_t IABR[8];          /*!< Offset: 0x200 (R/W) Interrupt Active bit Register */
    uint32_t RESERVED4[56];
    __IO uint8_t IP[240];           /*!< Offset: 0x300 (R/W) Interrupt Priority Register (8Bit wide) */
    uint32_t RESERVED5[644];
    __IO uint32_t STIR;             /*!< Offset: 0xE00 ( /W) Software Trigger Interrupt Register */
} NVIC_Type;
```

```
#define __IO volatile
```

6.2 NVIC控制器库函数

NVIC寄存器定义

- Core_m3.h中对NVIC控制器进行了声明

NVIC是一个指针

```
/* Memory mapping of Cortex-M3 Hardware */
#define SCS_BASE      (0xE000E000UL)           /*!< System Control Space Base Address */
#define ITM_BASE      (0xE0000000UL)           /*!< ITM Base Address */
#define DWT_BASE      (0xE0001000UL)           /*!< DWT Base Address */
#define TPI_BASE      (0xE0040000UL)           /*!< TPI Base Address */
#define CoreDebug_BASE (0xE000EDF0UL)          /*!< Core Debug Base Address */
#define SysTick_BASE   (SCS_BASE + 0x0010UL)    /*!< SysTick Base Address */
#define NVIC_BASE      (SCS_BASE + 0x0100UL)    /*!< NVIC Base Address */
#define SCB_BASE       (SCS_BASE + 0x0D00UL)    /*!< System Control Block Base Address */
#define SCB              ((SCB_Type *) SCB_BASE) /*!< SCB configuration struct */
#define SysTick          ((SysTick_Type *) SysTick_BASE) /*!< SysTick configuration struct */
#define NVIC            ((NVIC_Type *) NVIC_BASE) //NVIC configuration struct
```

6.2 NVIC控制器库函数

NVIC数据结构

- ST提供了NVIC标准库函数，头文件位misc.h，程序源代码位misc.c

NVIC_InitTypeDef结构体用于中断的初始化配置

typedef struct

{

uint8_t NVIC_IRQChannel; //Specifies the IRQ channel to be enabled or disabled.
 a value of @ref IRQn_Type

uint8_t NVIC_IRQChannelPreemptionPriority; // Specifies the pre-emption priority for the IRQ channel
 specified in NVIC_IRQChannel.
 between 0 and 15

uint8_t NVIC_IRQChannelSubPriority; //Specifies the subpriority level for the IRQ channel specified
 in NVIC_IRQChannel.
 between 0 and 15

FunctionalState NVIC_IRQChannelCmd; // Specifies whether the IRQ channel will be enabled
 ENABLE or DISABLE */

} NVIC_InitTypeDef;

6.2 NVIC控制器库函数

NVIC API函数

- NVIC_IRQChannel的取值

```
typedef enum IRQn
{
    /***** Cortex-M3 Processor Exceptions Numbers *****/
    NonMaskableInt_IRQn    = -14, /*!< 2 Non Maskable Interrupt          */
    MemoryManagement_IRQn  = -12, /*!< 4 Cortex-M3 Memory Management Interrupt */
    BusFault_IRQn           = -11, /*!< 5 Cortex-M3 Bus Fault Interrupt      */
    UsageFault_IRQn         = -10, /*!< 6 Cortex-M3 Usage Fault Interrupt    */
    SVC_IRQn                = -5,  /*!< 11 Cortex-M3 SV Call Interrupt      */
    DebugMonitor_IRQn       = -4,  /*!< 12 Cortex-M3 Debug Monitor Interrupt */
    PendSV_IRQn             = -2,  /*!< 14 Cortex-M3 Pend SV Interrupt      */
    SysTick_IRQn            = -1,  /*!< 15 Cortex-M3 System Tick Interrupt  */
    /***** STM32L specific Interrupt Numbers *****/
    WWDG_IRQn               = 0,    /*!< Window WatchDog Interrupt          */
    PVD_IRQn                = 1,    /*!< PVD through EXTI Line detection Interrupt */
    TAMPER_STAMP_IRQn       = 2,    /*!< Tamper and Time Stamp through EXTI Line Interrupts */
    RTC_WKUP_IRQn           = 3,    /*!< RTC Wakeup Timer through EXTI Line Interrupt */
    FLASH_IRQn              = 4,    /*!< FLASH global Interrupt            */
    RCC_IRQn                = 5,    /*!< RCC global Interrupt              */
    EXTI0_IRQn              = 6,    /*!< EXTI Line0 Interrupt              */
    EXTI1_IRQn              = 7,    /*!< EXTI Line1 Interrupt              */
    EXTI2_IRQn              = 8,    /*!< EXTI Line2 Interrupt              */
    EXTI3_IRQn              = 9,    /*!< EXTI Line3 Interrupt              */
    EXTI4_IRQn              = 10,   /*!< EXTI Line4 Interrupt              */
    DMA1_Channel1_IRQn      = 11,   /*!< DMA1 Channel 1 global Interrupt    */
    DMA1_Channel2_IRQn      = 12,   /*!< DMA1 Channel 2 global Interrupt    */
    DMA1_Channel3_IRQn      = 13,   /*!< DMA1 Channel 3 global Interrupt    */
    DMA1_Channel4_IRQn      = 14,   /*!< DMA1 Channel 4 global Interrupt    */
}
```

6.2 NVIC控制器库函数

NVIC API函数

- ST提供了NVIC标准库函数，头文件位misc.h，程序源代码位misc.c

以下是几个主要函数

函数名	描述
NVIC_PriorityGroupConfig	设置优先级分组，先占优先级和从优先级
NVIC_Init	根据NVIC_InitStruct中指定的参数初始化外设NVIC寄存器
NVIC_SetVectorTable	设置向量表的位置和偏移
NVIC_SystemLPConfig	选择系统进入低功耗模式的条件

6.2 NVIC控制器库函数

NVIC API函数

- NVIC_PriorityGroupConfig函数

函数功能：用于设置嵌套向量中断控制器的优先级分组。

函数原型：void NVIC_PriorityGroupConfig(uint32_t NVIC_PriorityGroup)

输入参数：NVIC_PriorityGroup_x x=0~4。

如果不执行该函数，默认的分组方式是分组0，抢占优先级为0，可以设置从优先级为0~15共16种情况。

应用示例：

NVIC_PriorityGroupConfig(NVIC_PriorityGroup_2)。

6.2 NVIC控制器库函数

NVIC API函数

- NVIC_Init函数

```
typedef struct
{
    uint8_t NVIC_IRQChannel; //设置中断通道
    uint8_t NVIC_IRQChannelPreemptionPriority; //设置响应优先级
    uint8_t NVIC_IRQChannelSubPriority; //设置抢占优先级
    FunctionalState NVIC_IRQChannelCmd; //使能/使能
} NVIC_InitTypeDef;
```

```
NVIC_InitTypeDef  NVIC_InitStructure;
NVIC_InitStructure.NVIC_IRQChannel = USART1_IRQn; //串口1中断
NVIC_InitStructure.NVIC_IRQChannelPreemptionPriority=1 ;// 抢占优先级为1
NVIC_InitStructure.NVIC_IRQChannelSubPriority = 2; // 子优先级位2
NVIC_InitStructure.NVIC_IRQChannelCmd = ENABLE; //IRQ通道使能
NVIC_Init(&NVIC_InitStructure); //根据上面指定的参数初始化NVIC寄存器
```



6.2 NVIC控制器库函数

NVIC API函数

- NVIC_SetVectorTable 函数

函数功能：于设置中断向量表的位置和偏移量。

函数原型：void NVIC_SetVectorTable(uint32_t NVIC_VectTab, uint32_t Offset)。

输入参数：

NVIC_VectTab用于指定中断向量表在RAM中还是Flash中，其取值为：

- NVIC_VectTab_RAM
- NVIC_VectTab_FLASH;

参数 Offset用于指定中断向量表的偏移量，一般设定为0。：

应用示例：

```
NVIC_SetVectorTable(NVIC_VectTab_FLASH, 0x0);
```

归纳和总结

◆ 抢占优先级 & 响应优先级区别：

- 高优先级的抢占优先级是可以打断正在进行的低抢占优先级中断的。
- 抢占优先级相同的中断，高响应优先级不可以打断低响应优先级的中断。
- 抢占优先级相同的中断，当两个中断同时发生的情况下，哪个响应优先级高，哪个先执行。
- 如果两个中断的抢占优先级和响应优先级都是一样的话，则看哪个中断先发生就先执行；

◆ 举例：

- 假定设置中断优先级组为2，然后设置
中断3(RTC中断)的抢占优先级为2，响应优先级为1。
中断6（外部中断0）的抢占优先级为3，响应优先级为0。
中断7（外部中断1）的抢占优先级为2，响应优先级为0。

那么这3个中断的优先级顺序为：中断7>中断3>中断6。

◆ 特别说明:

一般情况下，系统代码执行过程中，只设置一次中断优先级分组，比如分组**2**，设置好分组之后一般不会再改变分组。随意改变分组会导致中断管理混乱，程序出现意想不到的执行结果。

◆ 中断优先级分组函数:

void NVIC_PriorityGroupConfig(uint32_t NVIC_PriorityGroup);

```
void NVIC_PriorityGroupConfig(uint32_t NVIC_PriorityGroup)
{
    assert_param(IS_NVIC_PRIORITY_GROUP(NVIC_PriorityGroup));
    SCB->AIRCR = AIRCR_VECTKEY_MASK | NVIC_PriorityGroup;
}
```

```
NVIC_PriorityGroupConfig(NVIC_PriorityGroup_2);
```

◆ 中断优先级设置步骤

① 系统运行后先设置中断优先级分组。调用函数：

void NVIC_PriorityGroupConfig(uint32_t NVIC_PriorityGroup);

整个系统执行过程中，只设置一次中断分组。

② 针对每个中断，设置对应的抢占优先级和响应优先级：

void NVIC_Init(NVIC_InitTypeDef* NVIC_InitStruct);



6.3 EXTI外部中断控制器

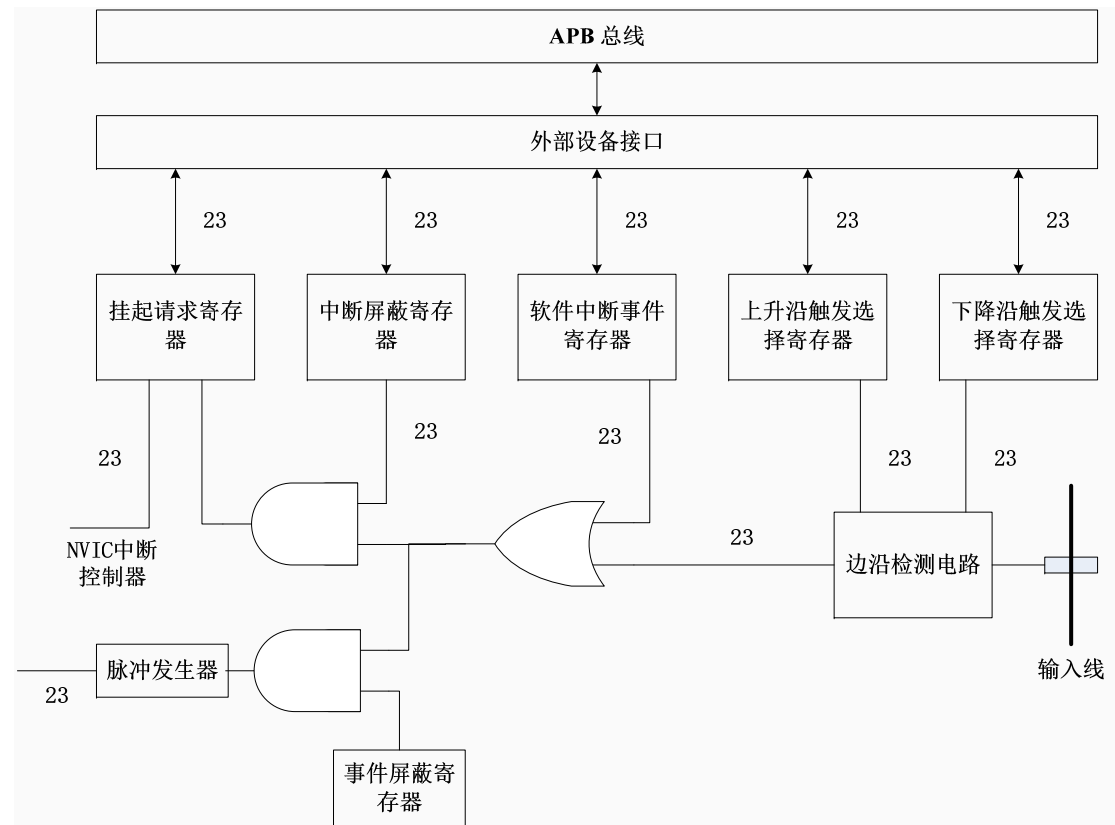
外部中断是由处理器的IO管脚产生的中断，由于处理器的IO管脚较多，使用也比较灵活，为了便于管理这些IO的中断，STM32L152提供了一个外部中断/事件控制器EXTI专门处理外部中断。

外部中断/事件控制器EXTI由**23**个产生中断/事件请求的边沿检测器组成，其中16用于IO中断，7个用于特殊事件处理。

- 每个输入线可以独立地配置输入类型（电平、脉冲），以及对应的触发事件（上升沿、下降沿或双边沿触发）。
- 每个输入线都可以独立地被触发和屏蔽。
- 每个中断线都有专用的状态位；

6.3 EXTI外部中断控制器

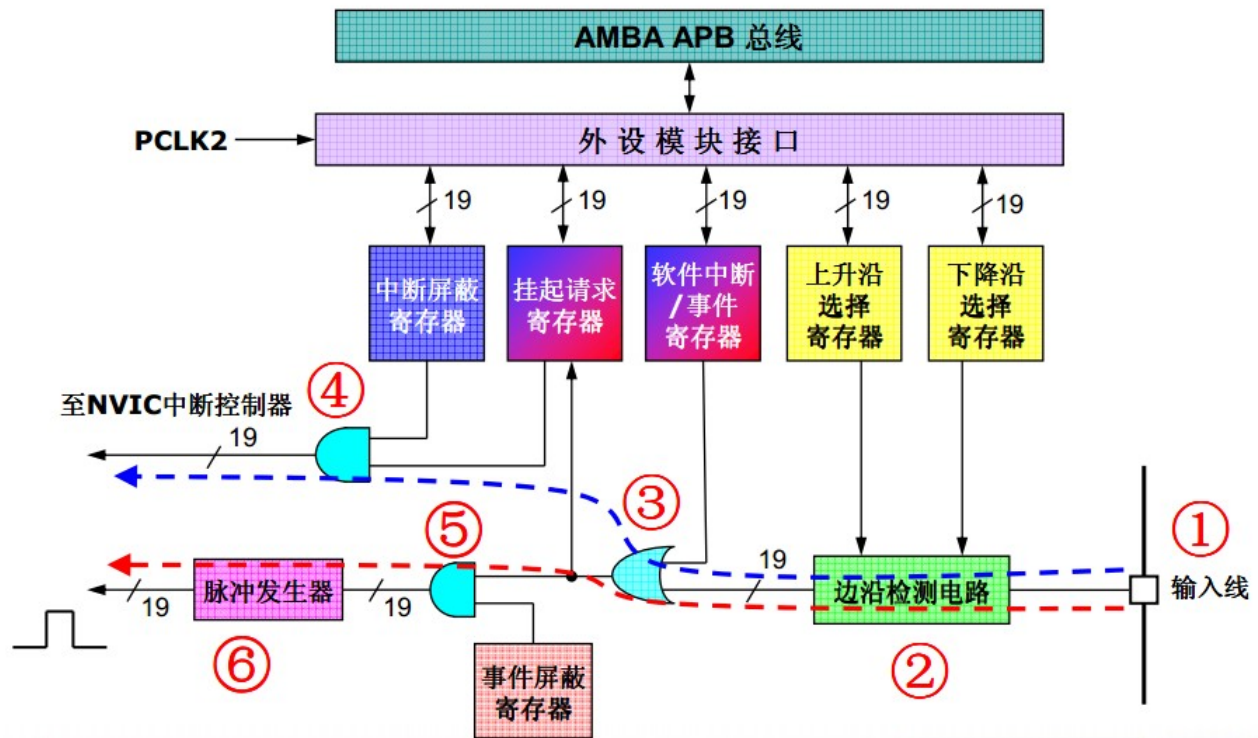
外部中断控制器的内部结构



6.3 EXTI外部中断控制器

• 中断和事件

事件屏蔽寄存器的控制;最后脉冲发生器的一个跳变的信号转变为一个单脉冲,输出到芯片中的其它功能模块



6.3 EXTI外部中断控制器

• 中断和事件

- 1) 如果要产生中断，必须对中断线进行配置并使能该中断线。根据需要的边沿检测设置2个触发寄存器，同时在中断屏蔽寄存器相应位写1来允许中断请求。当外部中断线上发生了期待的边沿时，将产生一个中断请求，对应的挂起位被置为1。
- 2) 如果要产生事件，必须对事件线进行配置并使能该事件线。根据需要的边沿检测设置2个触发寄存器，同时在事件屏蔽寄存器相应位写1来允许事件请求。当事件线上发生了需要的边沿时，将产生一个事件请求脉冲，对应的挂起位不被置1。
- 3) 通过软件向软件中断/事件寄存器写1，也可以产生中断/事件请求。



6.3 EXTI外部中断控制器

外部中断控制器寄存器

寄存器名称	地址偏移量	功能	复位值
中断屏蔽寄存器 EXTI_IMR	0x00	设定中断是否屏蔽	0x 0000 0000
EXTI事件屏蔽寄存器 (EXTI_EMR)	0x04	设定事件是否屏蔽	0x 0000 0000
EXTI 上升沿触发选择寄存器 (EXTI_RTSR)	0x08	配置中断/事件上升沿触发	0x 0000 0000
EXTI下降沿触发选择寄存器EXTI_FTSR	0x0C	配置中断/事件下降沿触发	0x 0000 0000
EXTI软件中断事件寄存器 EXTI_SWIE R	0x10	控制软件触发中断/事件	0x 0000 0000
EXTI挂起寄存器EXTI_PR	0x14	中断的挂起状态	0x xxxx xxxx
EXTI 中断系统配置寄存器 SYSCFG_EXTIRn		IO映射	

6.3 EXTI外部中断控制器

外部中断控制器寄存器

- **硬件中断选择过程：**
 - 1) 配置中断线的屏蔽位 (EXTI_IMR) ；
 - 2) 配置所选中断线的触发选择位 (EXTI_RTSTR和EXTI_FTSR) ；
 - 3) 配置对应到外部中断控制器 (EXTI) 的NVIC中断通道的使能和屏蔽位，使得中断线的请求可以被正确地响应。
- **硬件事件选择过程：**
 - 1) 配置事件线的屏蔽位 (EXTI_EMR) ；
 - 2) 配置所选事件线的触发选择位 (EXTI_RTSTR和EXTI_FTSR) 。
- **软件中断/事件选择过程：**
 - 1) 配置中断/事件线的屏蔽位 (EXTI_IMR和EXTI_EMR) ；
 - 2) 设置软件中断寄存器的请求位 (EXTI_SWIER) 。

6.4 EXTI外部中断控制器库函数

EXTI数据结构

- ST提供了EXTI标准库函数，头文件位stm32L1xx_exti.h，程序源代码位stm32L1xx_exti.c

EXTI_InitTypeDef 结构体用于外部中断的初始化配置

typedef struct

{

uint32_t EXTI_Line; // EXTI lines to be enabled or disabled, EXTI_Line

EXTIMode_TypeDef EXTI_Mode; // mode for the EXTI lines, EXTIMode_TypeDef

EXTITrigger_TypeDef EXTI_Trigger; //trigger signal active edge , EXTITrigger_TypeDef

FunctionalState EXTI_LineCmd; // ENABLE or DISABLE

}EXTI_InitTypeDef;

6.4 EXTI外部中断控制器库函数

EXTI API函数

- ST提供了EXTI标准库函数，头文件位stm32L1xx_exti.h，程序源代码位stm32L1xx_exti.c

函数名	描述
EXTI_DeInit	将EXTI控制器的寄存器复位
EXTI_Init	根据EXTI_InitStruct中指定的参数初始化外设EXTI寄存器
EXTI_StructInit	将EXTI_InitTypeDef 结构体变量的每个成员初始化为默认值
EXTI_GenerateSWInterrupt	产生一个软件中断
EXTI_GetFlagStatus	检查指定的EXTI线路标志位设置与否
EXTI_ClearFlag	清除EXTI线路挂起标志位
EXTI_GetITStatus	检查指定的EXTI线路触发请求发生与否
EXTI_ClearITPendingBit	清除EXTI线路挂起位



6.4 EXTI外部中断控制器库函数

EXTI API函数

- EXTI_DeInit函数

函数功能：将外设EXTI寄存器重设为缺省值

函数原型：void EXTI_DeInit(void)

应用示例：

```
EXTI_DeInit();
```

6.4 EXTI外部中断控制器库函数

EXTI API函数

- EXTI_Init函数

函数功能：将外设EXTI寄存器重设为缺省值

函数原型：void EXTI_Init(EXTI_InitTypeDef* EXTI_InitStructure)

输入参数：EXTI_InitStruct：指向结构EXTI_InitTypeDef的指针，包含了外设EXTI的配置信息

应用示例：

Enables external lines 12 and 14 interrupt generation on falling edge

```
EXTI_InitTypeDef EXTI_InitStructure;
```

```
EXTI_InitStructure.EXTI_Line = EXTI_Line12 | EXTI_Line14;
```

```
EXTI_InitStructure.EXTI_Mode = EXTI_Mode_Interrupt;
```

```
EXTI_InitStructure.EXTI_Trigger = EXTI_Trigger_Falling;
```

```
EXTI_InitStructure.EXTI_LineCmd = ENABLE;
```

```
EXTI_Init(&EXTI_InitStructure);
```

6.4 EXTI外部中断控制器库函数

EXTI API函数

- EXTI_StructInit函数

函数功能：把EXTI_InitStruct中的每一个参数按缺省值填入

函数原型：void EXTI_StructInit(EXTI_InitTypeDef*EXTI_InitStruct)

输入参数：EXTI_InitStruct：指向结构EXTI_InitTypeDef的指针默认值：

EXTI_Line = EXTI_LineNone

EXTI_Mode = EXTI_Mode_Interrupt

EXTI_Trigger = EXTI_Trigger_Falling

EXTI_LineCmd = DISABLE

应用示例：

```
EXTI_InitTypeDef EXTI_InitStructure;
```

```
EXTI_StructInit(&EXTI_InitStructure);
```

6.4 EXTI外部中断控制器库函数

EXTI API函数

- EXTI_GenerateSWInterrupt函数

函数功能：产生一个软件中断

函数原型：void EXTI_GenerateSWInterrupt(u32 EXTI_Line)

输入参数：EXTI_Line待使能或者失能的EXTI线路

应用示例：

```
EXTI_GenerateSWInterrupt(EXTI_Line6);
```

6.4 EXTI外部中断控制器库函数

EXTI API函数

- EXTI_GetFlagStatus 函数

函数功能：检查指定的EXTI线路标志位设置与否

函数原型：FlagStatus EXTI_GetFlagStatus(u32 EXTI_Line)

输入参数：待检查的EXTI线路标志位

返回值： EXTI_Line的状态（SET或者RESET）

应用示例：

```
FlagStatus EXTISatus;
```

```
EXTISatus = EXTI_GetFlagStatus(EXTI_Line8);
```

FlagStatus 枚举类型

6.4 EXTI外部中断控制器库函数

EXTI API函数

- EXTI_GetITStatus 函数

函数功能：检查指定的EXTI线路触发请求发生与否

函数原型：ITStatus EXTI_GetITStatus(u32 EXTI_Line)

输入参数：EXTI_Line待待检查EXTI线路的挂起位

返回值：EXTI_Line的状态（SET或者RESET）

应用示例：

```
ITStatus EXTISatus;
```

```
EXTISatus = EXTI_GetITStatus(EXTI_Line8);
```

ITStatus 枚举类型

6.4 EXTI外部中断控制器库函数

EXTI API函数

- EXTI_ClearFlag 函数

函数功能：清除EXTI线路挂起标志位

函数原型：void EXTI_ClearFlag(u32 EXTI_Line)

输入参数：EXTI_Line待清除的EXTI线路挂起标志位

应用示例：

```
EXTI_ClearFlag(EXTI_Line2);
```

6.4 EXTI外部中断控制器库函数

EXTI API函数

- EXTI_ClearITPendingBit 函数

函数功能：清除EXTI线路挂起位

函数原型：void EXTI_ClearITPendingBit (u32 EXTI_Line)

输入参数：EXTI_Line待清除的EXTI线路的挂起位

应用示例：

```
EXTI_ClearITpendingBit(EXTI_Line2);
```

6.4 EXTI外部中断控制器库函数

EXTI API函数

- SYSCFG_EXTILineConfig 函数

函数功能：选择GPIO管脚作为中断输入

函数原型：void SYSCFG_EXTILineConfig(uint8_t
EXTI_PortSourceGPIOx, uint8_t EXTI_PinSourcex)

输入参数1：GPIO管脚EXTI_PortSourceGPIOx, x的取值范围为(A..H)

输入参数2：配置的中断通EXTI_PinSourcex, x取值范围为(0..15).

应用示例：

```
SYSCFG_EXTILineConfig(EXTI_PortSourceGPIOA, EXTI_PinSource0);
```

6.4 EXTI外部中断控制器库函数

◆ 外部中断的一般配置步骤：

- ① 初始化IO口为输入。
GPIO_Init();
- ② 开启IO口复用时钟。
RCC_APB2PeriphClockCmd(RCC_APB2Periph_AFIO,ENABLE);
- ③ 设置IO口与中断线的映射关系。
void GPIO_EXTILineConfig();
- ④ 初始化线上中断，设置触发条件等。
EXTI_Init();
- ⑤ 配置中断分组（NVIC），并使能中断。
NVIC_Init();
- ⑥ 编写中断服务函数。
EXTIx_IRQHandler();
- ⑦ 清除中断标志位
EXTI_ClearITPendingBit();



6.5 中断使用示例

- 按键输入

通过连接PA0口的按键产生中断，来切换连接到PB7口的LED灯显示

```
GPIO_InitTypeDef GPIO_InitStructure;  
    // 外部中断控制器的初始化的结构体  
EXTI_InitTypeDef EXTI_InitStructure;  
    // 嵌套向量中断控制器的初始化的结构体  
NVIC_InitTypeDef NVIC_InitStructure;
```

实际工程案例展示